When someone feels sick, he or she goes to see the doctor. The patient does not know how much he will pay. I believe most of us have the similar difficulty. When we receive the bill from the hospital, it is usually a huge amount if we don't have insurance.

I am thinking, whether we can develop an app to solve this difficulty for everyone. I want to work on this project. Please let me know the feasibility. I also need a lot of data for training.

I'm checking what data sources and rules currently exist for healthcare price transparency so I can give you a grounded view of whether this app is realistic and what training data you'd actually need.

Thought for 22s ›

Yes — this is feasible, but only if you define the product carefully.

The easiest version is **not** "predict the exact final hospital bill for every person." That is much harder because the true amount depends on diagnosis changes, bundled services, clinician decisions, site of care, insurer contract details, deductibles, coinsurance, copays, prior authorization, out-of-network involvement, and coding after the visit. A more realistic first product is: **"given a procedure, location, insurer, and plan status, estimate a likely price range and expected out-of-pocket cost, with confidence bands and clear caveats."** That can be genuinely useful. Today, hospitals are required to post machine-readable price files and consumer-friendly displays for shoppable services, and CMS's 2026 updates added more allowed-amount detail such as median, 10th percentile, 90th percentile, and counts for certain historical allowed amounts. Uninsured or self-pay patients also have federal rights to request a Good Faith Estimate. Centers for Medi... +3

So the project is feasible, but the app should probably have **three stages**. Stage 1: a price transparency search engine that helps users compare hospitals and cash prices for common "shoppable" procedures. Stage 2: a personalized estimator that incorporates insurance status, deductible progress, and provider network information.

Stage 3: a navigation assistant that tells users what to ask for before care, including Good Faith Estimates or whether a service may trigger surprise-billing issues. That progression is practical because the raw data needed for Stage 1 is increasingly public, while Stages 2 and 3 require much better payer, claims, and benefit-design data.

Centers for Medi... +3

The biggest challenge is **data, not modeling**. Public hospital price-transparency files are helpful, but they are noisy, inconsistent across hospitals, and often difficult to standardize. They tell you standard charges, negotiated rates, cash prices, and some allowed-amount information, but they do not fully tell you what a specific patient will owe after deductible accumulation, plan rules, referrals, or downstream services. Medicare public datasets can help you learn charge and payment patterns at the provider and procedure level, but Medicare is only part of the market. All-payer claims databases can be much more valuable because they aggregate claims across multiple insurers, but access varies by state and is often restricted. CMS also requires payer claims and encounter data to be exposed through patient access APIs, which opens a future path for personalized estimation if a user connects their payer data.

Centers for Medi... +4

For training, you would ideally want data at several levels:

First, **reference pricing data**: hospital machine-readable files, hospital consumer-friendly shoppable-service pages, Medicare utilization/payment data, provider directories, hospital metadata, and geography. These are enough to build a baseline estimator and comparison tool. Centers for Medi... +3

Second, **claims-level data**: adjudicated claims with CPT/HCPCS/DRG codes, billed amount, allowed amount, patient responsibility, network status, insurer, plan type, site of care, and ZIP/state. This is the most important ingredient for a good personalized model, but it is the hardest to obtain. APCDs are one possible route; partnerships with self-funded employers, TPAs, payers, or providers are another. apcdcouncil.org +2

Third, **benefit-design data**: deductible, out-of-pocket maximum, coinsurance, copay rules, referral rules, exclusions, and accumulator status. Without this, you can estimate total allowed amount better than true patient out-of-pocket cost. CMS notes that payers are making claims and encounter data available by API, but benefit interpretation remains a product challenge. Centers for Medi... +1

Fourth, **episode context data**: many bills are not one line item. A surgery can involve facility fees, anesthesia, pathology, imaging, labs, assistant surgeons, and post-acute

care. The No Surprises Act materials themselves emphasize that estimates may involve multiple providers or facilities, and current enforcement has historically been imperfect for multi-provider coordination. That means your model should think in terms of **episodes of care**, not only individual procedure codes. Centers for Medi… +2

The good news is that you do **not** need a giant foundation model at the beginning. A strong first system could be built with structured data engineering plus classical prediction methods: code normalization, provider matching, geospatial features, benefit-rule engines, quantile regression, and uncertainty calibration. In fact, for this problem, a transparent model may be better than a black-box neural net because users will ask, “Why is this estimate so high?” The app should explain the estimate in plain language: hospital fee, professional fee, anesthesiology, expected insurer allowed amount, and likely patient share. The LLM part is best used later for explanation, plan-document parsing, and question answering, not as the core price engine. The public transparency ecosystem is also moving toward FHIR-based workflows for Good Faith Estimates and patient cost transparency, which is a good sign for future integration. Build FHIR +2

There are also major business and regulatory issues. You would need HIPAA-grade handling if you store identifiable health or claims data. You would need careful disclaimers because an estimate is not a guarantee. You would also need a product strategy for liability: the app should clearly distinguish between public posted prices, insurer-negotiated allowed amounts, and estimated patient responsibility. But these are manageable if you start as a transparency and decision-support tool rather than promising exact billing. The market need is real because the federal rules themselves are aimed at making prices more knowable before care. Centers for Medi… +2

My honest view: this is **very feasible as a startup or research prototype**, especially if your first target is one of these narrower cases:

1. uninsured/self-pay cash price estimator for common elective services,
2. insurer-specific comparison tool for a limited set of procedures,
3. employer-facing app that helps employees shop for imaging, labs, colonoscopy, childbirth, and other high-variance “shoppable” services,
4. post-bill audit tool that flags when a patient’s bill looks far outside local negotiated ranges.

The hardest version is “tell any patient exactly what their hospital bill will be.” The most winnable version is “give patients a reliable pre-care price range and show what drives it.”

A sensible roadmap would be:

- build a data pipeline that ingests hospital machine-readable files and standardizes codes, hospitals, and payers;
- focus on 20–50 common shoppable services first;
- create a range estimator by procedure, geography, and hospital;
- add insurance-plan logic later;
- only after that pursue claims partnerships for personalized out-of-pocket prediction.

If you want, I can turn this into a concrete startup plan with product scope, data sources, MVP features, and a 6-month technical roadmap.

Sources

go

I'm turning this into a practical plan: product scope, data pipeline, what you can build with public data, and where the biggest blockers are.

Thought for 14s

Here is a concrete way to think about it.

Your idea is feasible, but the winning version is **not** "tell any patient exactly what the final hospital bill will be." The winning version is: **before care, estimate a realistic price range and likely out-of-pocket cost, explain what drives the estimate, and help the patient compare providers.** That is aligned with the current U.S. transparency environment because hospitals now have to publish machine-readable files with standard charges, including discounted cash prices and payer-specific negotiated charges, and the 2026 rules added more detail for certain allowed-amount situations. CMS says enforcement of the updated 2026 hospital price transparency requirements began April 1, 2026. Centers for Medi... +3

The best first customer is probably one of these three groups: uninsured or self-pay patients shopping for elective care, insured patients comparing "shoppable" services like imaging or outpatient procedures, or employers trying to help employees avoid very expensive facilities for the same service. That is because hospitals must publish a

consumer-friendly display for shoppable services in addition to the full machine-readable file, so there is already a public-data foundation for an MVP. Centers for Medi… +2

A good MVP would do four things. First, search for a service in plain language, like “MRI knee without contrast” or “colonoscopy.” Second, map that to billing codes and nearby providers. Third, show a price range by hospital and a likely cash price or negotiated-price range. Fourth, explain uncertainty: “this estimate excludes complications, separate clinician bills, and services from other facilities.” That caveat matters because CMS’s Good Faith Estimate guidance itself says estimates are based on information known at the time and may miss items from other providers or facilities. Centers for Medi… +2

For data, you should think in layers. The first layer is **public hospital price transparency data**: machine-readable files, shoppable-service pages, hospital metadata, provider directories, and geography. This is enough to build a comparison engine and baseline estimator. The second layer is **claims data**: billed amount, allowed amount, patient responsibility, CPT/HCPCS/DRG codes, insurer, network status, site of care, and geography. This is what you need for real personalization. The third layer is **plan design data**: deductible, coinsurance, copays, out-of-pocket max, referral rules, and accumulator status. Without that third layer, you can estimate total price much better than exact patient liability. Centers for Medi… +4

The easiest way to start is with public data plus a narrower use case. For example, focus only on self-pay cash-price shopping for 20 to 50 common services in one state or metro area. Hospitals must expose discounted cash prices in the machine-readable files, and uninsured or self-pay patients also have federal rights around Good Faith Estimates. That means your first version can help users answer: “Where can I get this service, and what is a reasonable price before I go?” Centers for Medi… +2

If you want the product to become truly personalized, you will eventually need claims and payer data. APCDs are one possible path because they aggregate claims from multiple payer sources, although availability differs by state. CMS-regulated payers are also required to make a patient’s adjudicated claims and encounter data available through the Patient Access API, which suggests a future workflow where a user connects their insurer data to your app. That is promising, but it is a later-stage capability, not where I would start. apcdcouncil.org +5

Technically, you do not need a giant deep-learning model first. The core of the system is mostly data engineering and decision logic: normalize hospital files, standardize codes, map plain-language queries to procedures, de-duplicate providers, cluster similar

services, and estimate a price distribution using quantile models. For the first version, transparent models are actually better than black boxes because users need to know *why* the estimate is high. The AI layer can then sit on top to explain bills in plain language, parse insurance documents, and generate "questions to ask before you schedule." The standards world is already moving toward Good Faith Estimate and Advanced Explanation of Benefits workflows in FHIR, which is a good long-term sign for interoperability. Build FHIR +2

The biggest product risk is that a medical bill is often an **episode**, not one line item. Surgery may involve facility fees, anesthesia, pathology, imaging, labs, and clinician charges. CMS's Good Faith Estimate materials explicitly note that estimates may not include every connected item from another provider or facility. So your data model should eventually move from single procedure codes to episodes of care. In practice, that means you might begin with simple outpatient services first, then expand later to bundled episodes like childbirth, cataract surgery, colonoscopy, or joint replacement. Centers for Medi... +1

From a business standpoint, there is a real need here. Many people still do not know what care will cost until afterward, and the public policy direction is clearly toward pre-service transparency. But you should avoid promising exact billing. Position the product as a **decision-support and price-navigation tool**. That keeps the value strong while reducing legal and trust risk. If you later ingest protected health information or linked claims, then privacy, consent, and security become central product requirements. The HL7 and CMS interoperability materials also make clear that these workflows involve privacy and security expectations. Centers for Medi... +2

A very practical 6-month roadmap would look like this. Month 1: choose one market, such as Indiana outpatient imaging and GI procedures. Month 2: build an ingestion pipeline for hospital machine-readable files and normalize procedure codes, payer names, and hospital names. Month 3: create a service search experience that maps consumer queries to codes. Month 4: build a range estimator and ranking interface showing likely low, median, and high prices. Month 5: add explanation text, caveats, and a "what to ask the provider" workflow. Month 6: pilot with a small group of patients or employers and measure whether users choose lower-cost providers or request estimates before care. The 2026 CMS additions around median, 10th percentile, 90th percentile, and counts for certain allowed-amount disclosures are especially useful for building range-based estimates rather than point estimates. Centers for Medi... +2

My recommendation is: start with **self-pay and shoppable services**, not full insurance prediction. That version is feasible now, the data is more accessible, and the user value is immediate. Then, once the product works, add payer-linked personalization through claims APIs or partnerships.

I can turn this into a startup brief next: problem statement, MVP definition, data schema, modeling plan, and what your first dataset pipeline should look like.

Sources

go

I'm pulling together the startup brief around one winnable entry point and the concrete data/model stack needed for an MVP.

Thought for 8s

Here is a startup brief you can actually use.

The right company is not "AI that predicts every hospital bill exactly." The right company is **a healthcare price navigation platform** that estimates a likely pre-care price range, translates billing language into plain English, and helps patients compare options before they receive care. That positioning is realistic because hospitals already have to publish machine-readable files and consumer-friendly shoppable-service displays, and CMS began enforcing the new and revised 2026 hospital price transparency requirements on April 1, 2026. Centers for Medi... +2

I would frame the company around this problem statement: patients usually do not know what care will cost before they go, especially for imaging, outpatient procedures, lab work, and common elective services. Even when prices are technically public, the files are hard to find, inconsistent across hospitals, and not usable for ordinary people. For uninsured or self-pay patients, the No Surprises Act also created a Good Faith Estimate framework, but those estimates are still based only on information known at the time and may exclude unknown or unexpected costs. Centers for Medi... +2

So the company's first promise should be simple: **"Before you schedule care, we show you a realistic price range, likely cash option, and what questions to ask."** That is much more believable than "we tell you the exact final bill." The exact final bill is hard

because benefit balances can change, other claims may hit first, multiple providers may bill separately, and episodes often include related services beyond the main procedure. The HL7 Patient Cost Transparency workflow itself notes that estimated balances are based on known claims and do not account for other Good Faith Estimates, and it now supports multi-provider workflows precisely because healthcare pricing is fragmented. Build FHIR +2

For the MVP, I would choose one narrow entry point: **self-pay and shoppable services**. Build for MRI, CT, colonoscopy, ultrasound, mammogram, outpatient endoscopy, and a few common lab panels in one geography first. This is the cleanest wedge because hospitals must provide both a machine-readable file and a consumer-friendly display of shoppable services, and self-pay users are less dependent on insurer accumulator status. Centers for Medi... +1

Your first product could have five core features. A search box where the user types plain language like “knee MRI” or “colonoscopy.” A code-matching engine that maps that request to CPT/HCPCS or service bundles. A provider comparison page that shows hospitals and likely price ranges. An explanation layer that says what may or may not be included. And a visit-prep assistant that generates questions such as whether anesthesia, pathology, or facility fees are billed separately. The value is not only the number; it is the decision support around the number. CMS’s own Good Faith Estimate materials stress that estimates may omit unknown or unexpected costs and may involve multiple items and services. Centers for Medi... +1

The data stack should be built in layers.

Layer 1: public transparency data. This includes hospital machine-readable files, consumer-friendly shoppable-service pages, hospital metadata, location data, and provider directories. This layer is enough to build the first searchable comparison engine. CMS explicitly describes the machine-readable file and consumer-friendly display as the two required public-facing outputs. Centers for Medi... +1

Layer 2: claims data. This is what you eventually need for strong estimation. At minimum, you want procedure code, diagnosis grouping if available, billed amount, allowed amount, patient responsibility, payer, plan type, network status, provider, place of service, date, and geography. APCDs are one route because they collect claims from multiple payer sources, but access varies by state and often requires an application process. apcdcouncil.org +3

Layer 3: benefit and eligibility logic. This includes deductible, out-of-pocket maximum, coinsurance, copays, and current balances. Without this, you can estimate total allowed amount better than exact patient out-of-pocket cost. The patient cost transparency standards and AEOB workflow exist because the final patient share depends on payer-side information, not just provider-side posted prices. Build FHIR +3

That leads to the training question. At the beginning, you do **not** need a giant model or a massive private dataset. You can start with structured prediction and ranking. Public hospital data can power the search engine, normalization layer, and a baseline range estimator. The first model can be a quantile model or hierarchical pricing model that predicts a low, middle, and high likely price by service, facility, payer category, and location. The key is calibration and interpretability, not model complexity. Once you have claims data, you can train a second-stage model for expected allowed amount and patient share.

A practical schema for the first database would include hospital ID, NPI if available, hospital name, payer name, plan name if listed, code type, code value, description, negotiated amount, discounted cash price, de-identified min/max or percentiles if present, billing class, service setting, file date, and source URL. The 2026 CMS updates are especially useful because they added more detailed allowed-amount reporting elements for applicable circumstances, which helps you build range-based outputs instead of a single brittle estimate. Centers for Medi... +2

On the modeling side, I would split the system into three engines. The first is a **normalization engine** that standardizes hospital files, payer names, code formats, and service descriptions. The second is an **estimation engine** that outputs price ranges rather than point estimates. The third is an **explanation engine** that converts code-heavy outputs into patient language. Large language models are most useful in that third layer: searching in plain English, summarizing what a bill line means, and generating provider questions. They are not the main source of truth for the actual price number.

The main technical risk is service bundling. A colonoscopy may bring separate facility, physician, pathology, and anesthesia charges. An MRI might or might not include radiologist interpretation in the same billing flow. That means you will eventually want an **episode-of-care layer**, where the app knows that one consumer-facing service may map to several common claim components. The HL7 patient cost transparency guidance explicitly supports workflows where multiple providers contribute to one collective Good Faith Estimate, which matches this episode-based view. Build FHIR +1

The business model can start in one of three ways. A direct-to-consumer subscription is possible, but hard early. A better first path may be B2B2C: partner with employers, benefits navigators, or self-funded plans that want employees to choose lower-cost facilities. Another strong path is selling a white-labeled transparency widget to provider groups or digital health platforms. The best initial KPI is not “prediction error” alone. It is user action: did the user compare sites, request an estimate, or choose a lower-cost provider?

A sensible 6-month build plan would be this. In month 1, choose one market and 20 to 50 target services. In month 2, build the ingestion and normalization pipeline for public hospital files. In month 3, launch search and code mapping in plain language. In month 4, add a range estimator and provider ranking interface. In month 5, add the explanation layer and “what to ask” assistant. In month 6, run a pilot with real users and measure whether they successfully price-shop. That sequence is realistic because the public data is already there, even if messy. Centers for Medi... +2

My recommendation is very specific: start with **Indiana**, target **self-pay outpatient imaging and GI procedures**, and treat the first version as a **price comparison and estimate assistant**, not a universal bill predictor. That keeps the scope small enough to build, gives you real user value, and creates a path toward richer personalization once you obtain claims or payer integrations.

I can next turn this into a one-page investor-style concept note or a technical MVP spec with database tables, model inputs, and API design.

Sources

go

Thought for a couple of seconds

Here is a tight version you could use immediately.

Concept note

Working title: ClearCare Pricing

What it does: ClearCare Pricing helps patients understand the likely cost of care *before* they receive it. The product estimates a realistic price range for common services, compares nearby providers, translates medical billing language into plain English, and helps users ask the right pre-visit questions. The first version would focus on self-pay and other shoppable outpatient services, where public hospital pricing data is most usable. Hospitals are required to publish machine-readable files and consumer-friendly pricing displays, and CMS began enforcing the updated 2026 hospital price transparency requirements on April 1, 2026.

Problem: Most patients do not know what they will owe until after care. Public pricing data exists, but it is fragmented, inconsistent, and difficult for ordinary people to use. Even official Good Faith Estimates can exclude unknown or unexpected items and may involve multiple providers, which makes healthcare shopping much harder than shopping in ordinary markets.

Initial wedge: Start with self-pay outpatient imaging and GI procedures in one geography, such as Indiana. These are comparatively standardized, frequently shoppable, and often show large price variation across facilities. The MVP would cover services such as MRI, CT, ultrasound, mammogram, colonoscopy, and upper endoscopy. CMS requires hospitals to make standard charges public and to provide a consumer-friendly display for shoppable services.

Core user promise: “Before you schedule care, we show you a realistic price range, likely cash option, and what questions to ask.”

Why now: Policy and standards are finally moving in the right direction. CMS price-transparency rules are in force, the No Surprises Act created Good Faith Estimate workflows for uninsured or self-pay patients, and HL7 Da Vinci Patient Cost Transparency standards are building toward structured Good Faith Estimate and Advanced Explanation of Benefits exchanges.

Product features for MVP:

1. Plain-language service search, such as “knee MRI” or “colonoscopy.”
2. Code mapping to CPT/HCPCS/service bundles.
3. Provider comparison showing likely low, median, and high prices.
4. Clear caveats about what may be excluded.
5. Visit-prep assistant that suggests questions about anesthesia, pathology, facility fees, and separate clinician billing.

Business model: The best first model is likely B2B2C rather than pure direct-to-consumer. Likely early buyers are self-funded employers, benefits navigators, or health-navigation platforms that want employees to choose lower-cost sites of care. A later path is payer or provider integration.

Key risk: The exact final bill is often episode-based, not line-item based. A single service may involve separate professional, facility, pathology, lab, or anesthesia charges. The app should therefore present calibrated ranges and explanations, not promises of exact final amounts. CMS and HL7 materials both reflect this multi-provider, multi-service reality.

Technical MVP spec

1. Scope

Phase 1 target

- Geography: Indiana
- Services: 20–50 shoppable outpatient services
- User type: uninsured/self-pay first, insured later
- Output: price range, cash price if available, provider comparison, explanation

2. Data sources

Public data for MVP

- Hospital machine-readable files
- Hospital consumer-friendly shoppable-service pages
- Hospital metadata and locations
- Provider directories
- Code dictionaries for CPT/HCPCS/DRG/service descriptions

Hospitals must publish standard charges in a machine-readable file, including gross charges, payer-specific negotiated charges, discounted cash prices, and de-identified minimum and maximum negotiated charges where applicable. CMS's 2026 updates also added more detailed historical allowed-amount reporting elements in certain circumstances.

Later-stage data

- Claims/APCD data for allowed amount and patient responsibility
- Payer eligibility and benefit data

- Patient Access API-linked adjudicated claims
- Benefit design logic: deductible, coinsurance, copay, out-of-pocket max

APCD access varies by state, and payer-side data is what enables more personalized patient-responsibility estimates.

3. Core database tables

Hospitals

- hospital_id
- name
- address
- city
- state
- zip
- latitude
- longitude
- license/NPI if available
- source_url
- file_last_updated

Services

- service_id
- plain_language_name
- code_type
- code_value
- canonical_description
- service_category
- likely_episode_group

Rates

- rate_id
- hospital_id
- service_id
- payer_name
- plan_name if available
- billing_class
- negotiated_amount
- discounted_cash_price
- min_negotiated

- max_negotiated
- percentile_10 if available
- median if available
- percentile_90 if available
- data_source_type
- effective_date

Provider display mappings

- mapping_id
- user_query
- normalized_service_id
- confidence_score
- synonym_source

Episodes

- episode_id
- anchor_service_id
- common_component_service_ids
- notes_on_possible_exclusions

4. System architecture

Pipeline A: ingestion

- discover hospital file URLs
- download machine-readable files
- parse CSV/JSON/XML variants
- normalize payer names and code formats
- de-duplicate hospitals and services
- store standardized rates

Pipeline B: search and mapping

- convert plain-language queries to normalized services
- support synonyms like “MRI knee” → CPT/service bundle candidates
- return confidence-ranked mappings

Pipeline C: estimation

- generate a calibrated low/median/high estimate
- prefer cash price for self-pay flow
- fall back to negotiated or regional proxy ranges if direct service match is sparse

Pipeline D: explanation

- explain likely inclusions/exclusions
- generate questions for the provider
- summarize uncertainty in plain English

5. Modeling plan

For the first version, use transparent structured methods, not a heavy deep model.

Model 1: service matching

- hybrid rules + embedding retrieval over service descriptions
- output top likely code/service mappings

Model 2: price range estimation

- quantile regression or hierarchical model
- inputs: service, hospital, geography, facility type, payer category, source freshness
- outputs: p10, p50, p90 price estimates

Model 3: explanation layer

- LLM used only after structured retrieval
- explains the result, not generates the number
- examples:
  - “This estimate likely includes the facility charge but may not include pathology.”
  - “Ask whether anesthesia is billed separately.”

6. API design

GET /search

- input: plain-language query, ZIP, radius
- output: matched services with confidence

GET /compare

- input: service_id, ZIP, radius, self_pay=true/false
- output: provider list with low/median/high estimates and available cash prices

GET /explain

- input: service_id, provider_id
- output: likely included items, excluded items, patient questions

POST /feedback

- input: user correction on service match or displayed estimate usefulness
- output: stored for retraining and quality improvement

7. Evaluation metrics

Data quality

- parse success rate by hospital
- rate normalization coverage
- payer name standardization accuracy

Search quality

- top-1 and top-3 service mapping accuracy

Estimate quality

- median absolute percentage error where ground truth is available
- calibration of predicted price intervals
- coverage: fraction of true prices inside predicted range

Product

- provider comparison click-through
- estimate-to-scheduling action rate
- fraction of users who request a Good Faith Estimate afterward

8. Legal and trust design

The app should never promise the exact final bill. It should clearly label outputs as estimates and disclose that actual cost may vary due to additional services, clinician billing, insurance status, or complications. That is consistent with CMS Good Faith Estimate guidance and HL7 cost-transparency workflows.

9. 12-week build plan

Weeks 1–2

- choose target services
- identify Indiana hospitals and file endpoints
- define normalized schema

Weeks 3–5

- build ingestion and parser stack
- normalize service descriptions and payer names
- create search synonym layer

Weeks 6–8

- build comparison UI and basic quantile estimator
- add self-pay cash-price logic

- create explanation templates

Weeks 9–10

- test on real hospital files
- manual QA on 20–30 service/provider outputs
- improve search matching and exclusions

Weeks 11–12

- user testing
- refine outputs
- pilot with a small employer, student health navigator, or consumer group

---

What I think you should do next

Your best immediate move is to build a **data ingestion prototype**, not an app UI first. If you can successfully collect and normalize 25 Indiana hospital pricing files for 20 shoppable services, then the project is real. If that pipeline fails badly, you learn early where the real bottlenecks are.

After that, the next milestone is a simple tool that answers one question well:

“For this service near me, what is a realistic self-pay price range, and which facilities are cheaper?”

That is narrow, useful, and buildable.

I can next draft the actual database schema and Python project structure for the ingestion pipeline.

go

I’m drafting the actual build blueprint now: data schema, folder structure, ingestion flow, and the first scripts you’d implement to test whether the data pipeline is viable.

Thought for 8s ›

Below is the actual blueprint I would use for version 1.

1. Product boundary

Build **a price-estimate engine for self-pay shoppable hospital services**, not a universal final-bill predictor. That scope fits the public data that hospitals must publish: a machine-readable file for all items and services, plus a consumer-friendly display for shoppable services. CMS says the shoppable display must cover at least 300 shoppable services, or as many as the hospital provides if fewer, and include plain-language descriptions plus ancillary grouping. Center… +2

Your first question the system should answer is:

"For this service near this ZIP code, what is a realistic self-pay price range, and which facilities look cheaper?"

That is a strong MVP because discounted cash prices are explicitly part of hospital transparency requirements, and consumer-friendly shoppable displays are meant for services patients can schedule in advance. Centers for Medi… +1

---

2. Recommended tech stack

Use something simple and robust:

- **Language:** Python
- **Database:** PostgreSQL
- **ORM:** SQLAlchemy
- **API:** FastAPI
- **Queue / jobs:** Celery or just cron at first
- **Storage for raw files:** S3 or local object storage
- **Search:** PostgreSQL full text first, then Elasticsearch/OpenSearch later
- **Front end:** simple React or even Streamlit for prototype
- **LLM usage:** only for explanation and plain-language query mapping, never as the source of truth for prices

---

3. Project folder structure

```
│   │   │   ├── rules.py
│   │   │   ├── retrieval.py
│   │   │   └── ranking.py
│   │   ├── estimation/
│   │   │   ├── self_pay.py
│   │   │   ├── quantiles.py
│   │   │   └── episode_logic.py
│   │   └── explanation/
│   │       ├── templates.py
│   │       └── questions.py
│   └── utils/
│       ├── io.py
│       ├── hashing.py
│       └── text.py
└── tests/
    ├── test_parsers.py
    ├── test_normalization.py
    ├── test_matching.py
    └── test_compare_api.py
```

---

4. Database schema

Table: `hospitals`

```sql
CREATE TABLE hospitals (
    hospital_id UUID PRIMARY KEY,
    cms_ccn TEXT,
    npi TEXT,
    name TEXT NOT NULL,
    street TEXT,
    city TEXT,
    state TEXT,
    zip TEXT,
    latitude DOUBLE PRECISION,
    longitude DOUBLE PRECISION,
    website_url TEXT,
    source_page_url TEXT,
    active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

Table: `source_files`

Tracks each machine-readable file you download.

```
CREATE TABLE source_files (
    source_file_id UUID PRIMARY KEY,
    hospital_id UUID REFERENCES hospitals(hospital_id),
    file_url TEXT NOT NULL,
    local_path TEXT,
    format TEXT,
    checksum TEXT,
    file_size_bytes BIGINT,
    last_modified_remote TEXT,
    fetched_at TIMESTAMP,
    parsed_at TIMESTAMP,
    parse_status TEXT,
    parse_error TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);
```

Table: `services`

This is your canonical service catalog.

```
CREATE TABLE services (
    service_id UUID PRIMARY KEY,
    canonical_name TEXT NOT NULL,
    service_category TEXT,
    code_type TEXT,
    code_value TEXT,
    description TEXT,
    plain_language_name TEXT,
    is_target_service BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

Examples:

- MRI knee without contrast
- Colonoscopy diagnostic
- CT abdomen/pelvis with contrast

Table: `service_synonyms`

```
CREATE TABLE service_synonyms (
    synonym_id UUID PRIMARY KEY,
    service_id UUID REFERENCES services(service_id),
    synonym_text TEXT NOT NULL,
    source TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);
```

Examples:

- “knee MRI”
- “MRI of knee”
- “colon screening scope”

Table: `rates`

This is the main fact table.

```
CREATE TABLE rates (
    rate_id UUID PRIMARY KEY,
    source_file_id UUID REFERENCES source_files(source_file_id),
    hospital_id UUID REFERENCES hospitals(hospital_id),
    service_id UUID REFERENCES services(service_id),
    raw_description TEXT,
    raw_code_type TEXT,
    raw_code_value TEXT,
    payer_name_raw TEXT,
    payer_name_normalized TEXT,
    plan_name TEXT,
    billing_class TEXT,
    negotiated_type TEXT,
    gross_charge NUMERIC,
    discounted_cash_price NUMERIC,
    negotiated_amount NUMERIC,
    min_negotiated NUMERIC,
    max_negotiated NUMERIC,
    percentile_10 NUMERIC,
    median_amount NUMERIC,
    percentile_90 NUMERIC,
    setting TEXT,
```

Why include percentile fields? Because CMS’s 2026 policy changes added more historical allowed-amount detail in applicable situations, including statistics such as

median, 10th percentile, and 90th percentile, which is useful for range-based outputs.

Centers for Medi...

Table: `service_mappings`

This stores how raw hospital rows map to your canonical services.

```
CREATE TABLE service_mappings (
    mapping_id UUID PRIMARY KEY,
    hospital_id UUID REFERENCES hospitals(hospital_id),
    raw_description TEXT NOT NULL,
    raw_code_type TEXT,
    raw_code_value TEXT,
    matched_service_id UUID REFERENCES services(service_id),
    match_method TEXT,
    confidence_score DOUBLE PRECISION,
    reviewed_by_human BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT NOW()
);
```

Table: `episode_components`

This is optional at the start, but I would add it early.

```
CREATE TABLE episode_components (
    component_id UUID PRIMARY KEY,
    anchor_service_id UUID REFERENCES services(service_id),
    component_service_id UUID REFERENCES services(service_id),
    component_type TEXT,
    frequency_weight DOUBLE PRECISION,
    notes TEXT
);
```

Example:

- anchor: colonoscopy
- component: anesthesia
- component: pathology
- component: facility fee

This matters because shoppable-service displays may group ancillary services, and the broader PCT workflow is built around multi-provider and multi-service estimation.

Centers for Medi… +2

---

5. First target services

Do not start with hundreds. Start with maybe 20.

Imaging

- MRI brain without contrast
- MRI knee without contrast
- CT abdomen/pelvis with contrast
- CT head without contrast
- Screening mammogram
- Ultrasound abdomen complete

GI / outpatient procedures

- Colonoscopy diagnostic
- Colonoscopy screening
- Upper GI endoscopy

Labs

- CBC
- CMP
- lipid panel
- HbA1c
- TSH

Hospitals' consumer-friendly displays are intended for shoppable services that can be scheduled in advance, such as imaging, lab tests, outpatient visits, and bundled services like colonoscopy. Centers for Medi…

---

6. Ingestion pipeline

Step A: discover hospital file URLs

Create a seed list of Indiana hospitals and their websites. Then look for:

- `/price-transparency`
- `/standard-charges`

- machine-readable file links
- CSV / JSON / TXT / XML references

You can also use CMS resource materials and hospital website patterns as guidance, but the URLs are not centralized in one perfect directory. CMS provides machine-readable file guidance and validator tools for required layouts and data specifications.

Centers for Medi... +2

Step B: fetch files

For each hospital:

1. download file
2. compute checksum
3. store metadata
4. skip if unchanged

Step C: parse into staging rows

Hospital files are messy. Build a generic normalized intermediate object:

```
{
    "hospital_name": "...",
    "description": "...",
    "code_type": "...",
    "code_value": "...",
    "payer_name": "...",
    "plan_name": "...",
    "billing_class": "...",
    "gross_charge": ...,
    "discounted_cash_price": ...,
    "negotiated_amount": ...,
    "min_negotiated": ...,
    "max_negotiated": ...,
    "percentile_10": ...,
    "median_amount": ...,
    "percentile_90": ...,
    "setting": "...",
    "is_shoppable": True/False
}
```

Step D: normalize

Normalize:

- payer names: `Anthem BCBS`, `Anthem`, `Blue Cross Blue Shield Anthem` → one name
- service descriptions: lowercase, remove punctuation, strip modality noise
- code fields: standardize CPT/HCPCS/DRG string formatting
- hospitals: deduplicate name variants

Step E: map to canonical services

Use a cascade:

1. exact code match
2. rule-based text match
3. embedding retrieval over canonical services
4. human review for low-confidence cases

---

7. Core Python classes

Here is the skeleton I would use.

`models/service.py`

```
from sqlalchemy import Column, String, Text, Boolean, DateTime
from sqlalchemy.dialects.postgresql import UUID
from app.core.database import Base
import uuid
from datetime import datetime

class Service(Base):
    __tablename__ = "services"

    service_id = Column(UUID(as_uuid=True), primary_key=True, default=uuid
    canonical_name = Column(Text, nullable=False)
    service_category = Column(String)
    code_type = Column(String)
    code_value = Column(String)
    description = Column(Text)
    plain_language_name = Column(Text)
    is_target_service = Column(Boolean, default=False)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow)
```

`services/parsers/generic_parser.py`

```python
from typing import Iterable, Dict, Any

class GenericParser:
    def parse(self, path: str) -> Iterable[Dict[str, Any]]:
        raise NotImplementedError
```

`services/normalization/codes.py`

```python
def normalize_code(code_type: str | None, code_value: str | None) -> tuple
    if not code_type or not code_value:
        return code_type, code_value
    code_type = code_type.strip().upper()
    code_value = code_value.strip().upper().replace(".", "")
    return code_type, code_value
```

`services/matching/rules.py`

```python
def rule_match_service(description: str) -> str | None:
    desc = description.lower()
    if "colonoscopy" in desc:
        return "COLONOSCOPY"
    if "mri" in desc and "knee" in desc:
        return "MRI_KNEE_WO"
    if "mammogram" in desc:
        return "SCREENING_MAMMOGRAM"
    return None
```

---

8. First API endpoints

`/search`

Input:

```json
{
  "query": "knee MRI",
```

```
  "zip_code": "47906",
  "radius_miles": 50
}
```

Output:

```
{
  "matches": [
    {
      "service_id": "uuid",
      "plain_language_name": "MRI knee without contrast",
      "confidence": 0.94
    }
  ]
}
```

`/compare`

Input:

```
{
  "service_id": "uuid",
  "zip_code": "47906",
  "radius_miles": 50,
  "self_pay": true
}
```

Output:

```
{
  "service": "MRI knee without contrast",
  "providers": [
    {
      "hospital_name": "Example Hospital",
      "distance_miles": 12.1,
      "cash_price": 480.00,
      "low_estimate": 450.00,
      "median_estimate": 480.00,
      "high_estimate": 650.00,
      "notes": ["Radiologist reading may be billed separately."]
    }
```

```
  ]
}
```

`/explain`

Input:

```JSON
{
  "service_id": "uuid",
  "hospital_id": "uuid"
}
```

Output:

```JSON
{
  "plain_english": "This estimate likely covers the hospital service itsel
  "questions_to_ask": [
    "Is the radiologist fee included?",
    "Are there any separate facility or physician charges?",
    "Can I request a Good Faith Estimate in writing?"
  ]
}
```

---

9. Estimation logic for version 1

For self-pay, keep it simple.

Logic:

1. If discounted cash price exists for exact matched service at hospital, use it.
2. If several exact-like rows exist, show min / median / max.
3. If no exact cash price exists:
   - use shoppable-service package price if available
   - otherwise infer from nearby hospitals for the same service
   - otherwise show “insufficient direct data”

Range generation:

- `low_estimate`: 25th percentile of matched prices

- `median_estimate`: median
- `high_estimate`: 75th or 90th percentile depending on density

This is better than forcing a single value.

---

10. What to build first, in order

Week 1

Create:

- `hospitals_indiana.yaml`
- `services.csv` with 20 target services
- parser skeleton
- database tables

Week 2

Build:

- hospital file discovery script
- fetcher
- `source_files` table population

Week 3

Build:

- one CSV parser
- one JSON parser
- staging output
- payer normalization dictionary

Week 4

Build:

- service matching rules
- first 5 canonical services
- compare API returning raw matched prices

Week 5

Build:

- range estimation

- ZIP/radius filtering
- basic UI

Week 6

Build:

- explanation templates
- provider questions
- manual QA dashboard

---

11. Immediate proof-of-feasibility milestone

Do **not** start by building the full consumer app.

Start with this milestone:

Ingest and normalize 25 Indiana hospital machine-readable files for 10 services.

Success criteria:

- at least 70% parse successfully
- at least 60% of target services can be matched for at least 10 hospitals
- discounted cash prices appear often enough to power a comparison view

If you hit that, the idea is real enough to keep going.

---

12. Biggest practical issue you will hit

The hardest technical problem early is not ML. It is **file inconsistency**:

- different file formats
- weird hospital naming
- payer name chaos
- raw descriptions that do not cleanly map to one service
- shoppable bundles versus individual line items

CMS has published machine-readable file guidance and validator materials, which helps, but real-world normalization will still be a major engineering task. Centers for Medi... +2

---

13. My recommendation

Your next concrete move should be:

build the ingestion prototype for Indiana hospitals first.

Once that works, then build `/search` and `/compare`.

If you want, I'll write the first actual files next:
`services.csv`, `hospitals_indiana.yaml`, and `discover_hospital_files.py`.

Sources